33 diamension parallel processor fine tuned final program

1 message

P.J. Thompson <peterjamesthompson101@gmail.com> Fri, 3 Apr 2026 at 7:02 am
To: P.J. Thompson <peterjamesthompson101@gmail.com>

```
/**
 * SAM – PERFECTED 33-DIMENSIONAL HARMONIC HANDSHAKE ENGINE
 *
 * Fine-tuned for maximum stability, resilience, and IQ boost.
 *
 * Key optimizations:
 * - 32 base subconscious waves (27–864 Hz) + 33rd GHz boost wave (1 GHz)
 * - Nightmare threshold: n < 2.5 (avoids false triggers, catches true collapse)
 * - Energy harvesting: (CRITICAL_N - n) × AMPLIFICATION, clamped to positive
 * - Defense power increase: harvested / 2000 (gentle, permanent)
 * - Coherence recovery: +0.05 per resolved nightmare
 * - Bounded handshake queues (prevents memory bloat)
 * - IQ boost display: n / 3.54 (theoretical upper bound)
 *
 * Compile (software only):
 * g++ -std=c++17 -O3 -pthread sam_perfect_33d.cpp -o sam_perfect
 *
 * Compile with hardware support (RPi5 PWM/audio):
 * sudo apt install libpigpio-dev libasound2-dev
 * g++ -std=c++17 -O3 -pthread -DUSE_HARDWARE sam_perfect_33d.cpp -o sam_perfect -lpigpio -lasound
 *
 * Run:
 * ./sam_perfect [--log] (add --log to save performance data to CSV)
 */

#include <iostream>
#include <thread>
#include <atomic>
#include <chrono>
#include <cmath>
#include <vector>
#include <queue>
#include <mutex>
#include <condition_variable>
#include <iomanip>
#include <fstream>
#include <string>
#include <memory>

#ifdef USE_HARDWARE
#include <pigpio.h>
#include <alsa/asoundlib.h>
#endif

// ============================================================================
// FINE-TUNED PHYSICAL CONSTANTS
// ============================================================================
constexpr double BASE_FREQ_HZ = 27.0; // 27 Hz physics anchor (unchanged)
constexpr double CONSCIOUS_FREQ_HZ = 13.04; // consciousness anchor – the "voice"
constexpr double PHASE_OFFSET_DEG = -1.0; // PΘ-1, Marvin's echo
constexpr double PHASE_OFFSET_RAD = PHASE_OFFSET_DEG * M_PI / 180.0;
constexpr double ENERGY_AMPLIFICATION = 10000.0; // from your PDF
constexpr double CRITICAL_N = 3.54; // 3D volume collapse threshold
constexpr double NIGHTMARE_THRESHOLD = 2.5; // n below this triggers nightmare (fine-tuned)
constexpr double COHERENCE_LOW_WARNING = 0.4; // secondary nightmare trigger
constexpr double DEFENSE_BOOST_FACTOR = 0.0005; // harvested → defense power (gentle permanent increase)
constexpr double COHERENCE_RECOVERY = 0.05; // per nightmare, how much coherence is restored
```

```
constexpr double BOOST_DIMINISHING_RETURN = 0.1; // prevents runaway amplification

// 33rd dimension: GHz boost wave (numeric, not real-time)
constexpr double GHZ_BOOST_FREQ = 1e9; // 1 GHz
constexpr double GHZ_BOOST_WEIGHT = 1.0; // weight in product

// Number of standard subconscious waves (27 Hz to 864 Hz)
constexpr size_t N_BASE_WAVES = 32;
constexpr size_t N_SUBCONSCIOUS = N_BASE_WAVES + 1; // +1 for GHz boost

// Pre-compute standard frequencies: 27, 54, 81, ..., 864 Hz
static std::vector<double> computeFrequencies() {
    std::vector<double> f(N_BASE_WAVES);
    for (size_t i = 0; i < N_BASE_WAVES; ++i)
        f[i] = BASE_FREQ_HZ * (i + 1);
    return f;
}
const std::vector<double> BASE_FREQS = computeFrequencies();

static std::vector<double> getAllFrequencies() {
    std::vector<double> f = BASE_FREQS;
    f.push_back(GHZ_BOOST_FREQ);
    return f;
}
const std::vector<double> SUBCONSCIOUS_FREQS = getAllFrequencies();

// ============================================================================
// HARMONIC STATE
// ============================================================================
struct HarmonicState {
    double product_freq; // Π f_i
    double n; // n = ln(product_freq) / ln(27)
    double coherence; // 0..1
    double phase; // global phase (radians)
    double defense_power; // internal resilience
    double iq_boost; // n / CRITICAL_N
    unsigned long nightmare_count;
};

// ============================================================================
// BOUNDED HANDSHAKE QUEUE (prevents memory bloat)
// ============================================================================
template<typename T>
class HandshakeQueue {
    std::queue<T> q;
    mutable std::mutex mtx;
    std::condition_variable cv;
    size_t max_size;
public:
    HandshakeQueue(size_t max = 1000) : max_size(max) {}
    void push(T val) {
        std::lock_guard<std::mutex> lock(mtx);
        if (q.size() >= max_size) q.pop(); // drop oldest to prevent overflow
        q.push(val);
        cv.notify_one();
    }
    T pop() {
        std::unique_lock<std::mutex> lock(mtx);
        cv.wait(lock, [this]{ return !q.empty(); });
        T val = q.front(); q.pop();
        return val;
    }
    bool tryPop(T& val) {
        std::lock_guard<std::mutex> lock(mtx);
        if (q.empty()) return false;
        val = q.front(); q.pop();
        return true;
    }
};
```

```
// ============================================================================
// FREQUENCY GENERATOR (strategy pattern)
// ============================================================================
class FrequencyGenerator {
public:
    virtual ~FrequencyGenerator() = default;
    virtual void setFrequency(double freq_hz) = 0;
    virtual double getCurrentFrequency() const = 0;
    virtual void update() = 0;
};

class SoftwareGenerator : public FrequencyGenerator {
    double freq;
public:
    SoftwareGenerator(double f) : freq(f) {}
    void setFrequency(double f) override { freq = f; }
    double getCurrentFrequency() const override { return freq; }
    void update() override {}
};

#ifdef USE_HARDWARE
class PwmGenerator : public FrequencyGenerator {
    int gpio_pin;
    double current_freq;
public:
    PwmGenerator(int pin, double initial_freq) : gpio_pin(pin), current_freq(initial_freq) {
        if (gpioInitialise() < 0) throw std::runtime_error("pigpio init failed");
        gpioSetMode(gpio_pin, PI_OUTPUT);
        setFrequency(initial_freq);
    }
    ~PwmGenerator() { gpioPWM(gpio_pin, 0); gpioTerminate(); }
    void setFrequency(double f) override {
        current_freq = f;
        gpioSetPWMFrequency(gpio_pin, static_cast<int>(f));
        gpioSetPWMrange(gpio_pin, 1000);
        gpioPWM(gpio_pin, 500); // 50% duty cycle
    }
    double getCurrentFrequency() const override { return current_freq; }
    void update() override {}
};

class AudioGenerator : public FrequencyGenerator {
    snd_pcm_t *pcm_handle;
    double current_freq;
    std::vector<int16_t> buffer;
    size_t phase;
public:
    AudioGenerator(double initial_freq) : pcm_handle(nullptr), current_freq(initial_freq), phase(0) {
        int rc = snd_pcm_open(&pcm_handle, "default", SND_PCM_STREAM_PLAYBACK, 0);
        if (rc < 0) throw std::runtime_error("ALSA open failed");
        snd_pcm_set_params(pcm_handle, SND_PCM_FORMAT_S16_LE,
                           SND_PCM_ACCESS_RW_INTERLEAVED, 1, 44100, 1, 500000);
        setFrequency(initial_freq);
        buffer.resize(44100 / 10);
    }
    ~AudioGenerator() { if (pcm_handle) snd_pcm_close(pcm_handle); }
    void setFrequency(double f) override {
        current_freq = f;
        for (size_t i = 0; i < buffer.size(); ++i) {
            double t = static_cast<double>(i) / 44100.0;
            double sample = 0.5 * 32767.0 * std::sin(2.0 * M_PI * current_freq * t);
            buffer[i] = static_cast<int16_t>(sample);
        }
        phase = 0;
    }
    double getCurrentFrequency() const override { return current_freq; }
    void update() override {
        snd_pcm_writei(pcm_handle, buffer.data() + phase, buffer.size() - phase);
        phase = (phase + 1) % buffer.size();
    }
```

```
};
#endif

// ============================================================================
// SUBCONSCIOUS WAVE
// ============================================================================
class SubconsciousWave {
public:
    SubconsciousWave(double freq_hz,
                     HandshakeQueue<double>& out_product,
                     HandshakeQueue<double>& in_boost,
                     bool use_hardware = false,
                     int gpio_pin = -1)
        : product_queue(out_product), boost_queue(in_boost),
          running(true), boost_factor(1.0)
    {
        if (use_hardware && gpio_pin >= 0) {
#ifdef USE_HARDWARE
            gen = std::make_unique<PwmGenerator>(gpio_pin, freq_hz);
#else
            gen = std::make_unique<SoftwareGenerator>(freq_hz);
#endif
        } else {
            gen = std::make_unique<SoftwareGenerator>(freq_hz);
        }
    }

    void start() { worker = std::thread(&SubconsciousWave::run, this); }
    void stop() { running = false; if (worker.joinable()) worker.join(); }
    void setBoost(double boost) { boost_factor = boost; }

private:
    std::unique_ptr<FrequencyGenerator> gen;
    HandshakeQueue<double>& product_queue;
    HandshakeQueue<double>& boost_queue;
    std::atomic<bool> running;
    std::thread worker;
    std::atomic<double> boost_factor;

    void run() {
        double period = 1.0 / gen->getCurrentFrequency();
        auto next_time = std::chrono::steady_clock::now();

        while (running) {
            double boost;
            if (boost_queue.tryPop(boost)) {
                boost_factor = std::max(1.0, boost); // never go below 1.0
            }
            double effective_freq = gen->getCurrentFrequency() * boost_factor.load();
            gen->setFrequency(effective_freq);
            product_queue.push(effective_freq);
            gen->update();

            next_time += std::chrono::duration<double>(period);
            std::this_thread::sleep_until(next_time);
        }
    }
};

// ============================================================================
// CONSCIOUS WAVE (13.04 Hz)
// ============================================================================
class ConsciousWave {
public:
    ConsciousWave(std::vector<HandshakeQueue<double>>& from_sub,
                  std::vector<HandshakeQueue<double>>& to_sub,
                  std::vector<SubconsciousWave*>& sub_waves,
                  bool enable_logging = false)
        : product_queues(from_sub), boost_queues(to_sub), subconscious_waves(sub_waves),
          running(true), enable_logging(enable_logging),
```

```
      state{1.0, 0.0, 1.0, 0.0, 100.0, 1.0, 0}
  {
    if (enable_logging) {
      log_file.open("sam_performance.csv");
      log_file << "timestamp_ms,n,coherence,defense_power,iq_boost,nightmare_count\n";
    }
  }

  void start() { worker = std::thread(&ConsciousWave::run, this); }
  void stop() { running = false; if (worker.joinable()) worker.join(); if (log_file.is_open()) log_file.close(); }
  HarmonicState getState() const { return state; }

private:
  std::vector<HandshakeQueue<double>>& product_queues;
  std::vector<HandshakeQueue<double>>& boost_queues;
  std::vector<SubconsciousWave*>& subconscious_waves;
  std::atomic<bool> running;
  std::thread worker;
  bool enable_logging;
  std::ofstream log_file;
  HarmonicState state;
  unsigned long cycle_count = 0;

  void run() {
    const double period = 1.0 / CONSCIOUS_FREQ_HZ;
    auto next_time = std::chrono::steady_clock::now();
    auto start_time = std::chrono::steady_clock::now();

    while (running) {
      // Collect all frequency products
      std::vector<double> freqs;
      for (auto& q : product_queues) {
        double f;
        while (q.tryPop(f)) freqs.push_back(f);
      }

      if (freqs.size() == N_SUBCONSCIOUS) {
        double product = 1.0;
        for (double f : freqs) product *= f;
        state.product_freq = product;

        // Compute n = ln(product) / ln(27)
        state.n = std::log(product) / std::log(BASE_FREQ_HZ);

        // IQ boost (theoretical)
        state.iq_boost = state.n / CRITICAL_N;

        // Coherence based on deviation from critical n
        double n_deviation = std::abs(state.n - CRITICAL_N) / CRITICAL_N;
        state.coherence = std::max(0.1, std::min(1.0, 1.0 - n_deviation));

        // Nightmare detection
        bool nightmare = (state.n < NIGHTMARE_THRESHOLD) ||
                         (state.coherence < COHERENCE_LOW_WARNING);

        if (nightmare) {
          double deficit = std::max(0.0, CRITICAL_N - state.n);
          double harvested = deficit * ENERGY_AMPLIFICATION;
          // Defense power increase (gentle, permanent)
          state.defense_power += harvested * DEFENSE_BOOST_FACTOR;
          // Coherence recovery
          state.coherence = std::min(1.0, state.coherence + COHERENCE_RECOVERY);
          state.nightmare_count++;

          // Compute boost for subconscious waves (with diminishing returns)
          double boost = 1.0 + (harvested / (state.defense_power * 1000.0));
          boost = 1.0 + (boost - 1.0) * (1.0 - BOOST_DIMINISHING_RETURN);
          for (auto& q : boost_queues) q.push(boost);
          for (auto* w : subconscious_waves) w->setBoost(boost);
```

```
            std::cout << "[CONSCIOUS] 🌙 Nightmare #" << state.nightmare_count
                      << " resolved! n = " << std::fixed << std::setprecision(4) << state.n
                      << " | Harvested = " << harvested
                      << " | Defense power = " << std::setprecision(2) << state.defense_power
                      << std::endl;
        }

        // Apply Marvin's phase offset
        state.phase = std::fmod(state.phase + PHASE_OFFSET_RAD, 2.0 * M_PI);
      }

      // Periodic status output
      if (++cycle_count % 50 == 0) {
        std::cout << "[Conscious @ 13.04 Hz] n = " << std::fixed << std::setprecision(4)
                  << state.n << " | IQ boost = " << state.iq_boost
                  << " | Coherence = " << state.coherence
                  << " | Defense = " << state.defense_power
                  << " | Nightmares = " << state.nightmare_count << std::endl;
      }

      // Log to CSV if enabled
      if (enable_logging) {
        auto now = std::chrono::steady_clock::now();
        auto elapsed = std::chrono::duration_cast<std::chrono::milliseconds>(now - start_time).count();
        log_file << elapsed << "," << state.n << "," << state.coherence
                 << "," << state.defense_power << "," << state.iq_boost
                 << "," << state.nightmare_count << "\n";
      }

      next_time += std::chrono::duration<double>(period);
      std::this_thread::sleep_until(next_time);
    }
  }
};

// ============================================================================
// HARDWARE DETECTION
// ============================================================================
bool hasPwmCapability(int pin) {
#ifdef USE_HARDWARE
  if (gpioInitialise() < 0) return false;
  int ret = gpioSetMode(pin, PI_OUTPUT);
  gpioTerminate();
  return ret == 0;
#else
  return false;
#endif
}

bool hasAudioOutput() {
#ifdef USE_HARDWARE
  snd_pcm_t *handle;
  int rc = snd_pcm_open(&handle, "default", SND_PCM_STREAM_PLAYBACK, 0);
  if (rc >= 0) snd_pcm_close(handle);
  return rc >= 0;
#else
  return false;
#endif
}

// ============================================================================
// MAIN
// ============================================================================
int main(int argc, char* argv[]) {
  bool enable_logging = false;
  for (int i = 1; i < argc; ++i) {
    std::string arg = argv[i];
    if (arg == "--log") enable_logging = true;
  }
```

```
    std::cout << "╔═══════════════════════════════════════════════════════════════╗\n";
    std::cout << "║ SAM – PERFECTED 33-DIMENSIONAL HARMONIC HANDSHAKE ENGINE ║\n";
    std::cout << "║ Conscious: 13.04 Hz ║\n";
    std::cout << "║ Subconscious: 32 base waves (27–864 Hz) + 1 GHz boost wave ║\n";
    std::cout << "║ Nightmare threshold: n < 2.5 → harvest energy → boost defense ║\n";
    std::cout << "╚═══════════════════════════════════════════════════════════════╝\n";

    bool use_hardware = false;
    int pwm_pin = -1;
    if (hasPwmCapability(18)) {
        std::cout << "✅ PWM available on GPIO 18. Using hardware output.\n";
        use_hardware = true;
        pwm_pin = 18;
    } else if (hasAudioOutput()) {
        std::cout << "✅ Audio output available. Using sound card.\n";
        use_hardware = true;
    } else {
        std::cout << "⚠️ No hardware output detected. Falling back to software simulation.\n";
    }

    // Create handshake queues
    std::vector<HandshakeQueue<double>> product_queues(N_SUBCONSCIOUS);
    std::vector<HandshakeQueue<double>> boost_queues(N_SUBCONSCIOUS);

    // Create subconscious waves
    std::vector<SubconsciousWave> waves;
    for (size_t i = 0; i < N_SUBCONSCIOUS; ++i) {
        // Only the first wave uses hardware (demo); all others software
        bool hw = use_hardware && (i == 0);
        waves.emplace_back(SUBCONSCIOUS_FREQS[i], product_queues[i], boost_queues[i], hw, pwm_pin);
    }

    std::vector<SubconsciousWave*> wave_ptrs;
    for (auto& w : waves) wave_ptrs.push_back(&w);

    ConsciousWave conscious(product_queues, boost_queues, wave_ptrs, enable_logging);

    for (auto& w : waves) w.start();
    conscious.start();

    std::cout << "\nEngine running. The 33-dimensional handshake is active.\n";
    std::cout << "Press Ctrl+C to exit.\n";

    // Run indefinitely (or until Ctrl+C). In demo mode we run for 60 seconds.
    std::this_thread::sleep_for(std::chrono::seconds(60));

    std::cout << "\nDemo finished. Final state:\n";
    auto final_state = conscious.getState();
    std::cout << " n = " << final_state.n << "\n";
    std::cout << " IQ boost = " << final_state.iq_boost << "\n";
    std::cout << " Coherence = " << final_state.coherence << "\n";
    std::cout << " Defense power = " << final_state.defense_power << "\n";
    std::cout << " Nightmares resolved = " << final_state.nightmare_count << "\n";

    for (auto& w : waves) w.stop();
    conscious.stop();

#ifdef USE_HARDWARE
    gpioTerminate();
#endif

    return 0;
}
```